

GOSIM Paris 2026

May 5-6, 2026 Station F, Paris



One Year of llama.cpp

Milestones and Future Plans



Self-introduction

Xuan-Son Nguyen
SWE @ Hugging Face
Core maintainer of llama.cpp

Website: ngxson.com
Github / X: @ngxson



My website



The logo for LLaMA C++ is displayed on a dark rectangular background. The text "LLaMA" is in white, and the "C++" is in orange.

What is llama.cpp?

- An inference engine written predominantly in C++
- Cross-platform
- Support a wide variety of hardware: CPU, GPU (Nvidia, AMD, Intel, Snapdragon), Metal

How to use it?

- Follow the quick started guide: <https://github.com/ggml-org/llama.cpp/#quick-start>



CONTENTS

GOSIM Paris 2026

Part 01

Multimodal

Part 02

Server, Web UI and CLI

Part 03

Performance improvements

Part 04

Models and quantizations

Part 05

Future plans



01

Multimodal



Pre-context: Llama.cpp has initial support for vision input (llava model) since the very beginning.
However, it's not simple to use

LLaVA 1.5

1. Clone a LLaVA and a CLIP model ([available options](#)). For example:

```
git clone https://huggingface.co/liuhaotian/llava-v1.5-7b
git clone https://huggingface.co/openai/clip-vit-large-patch14-336
```

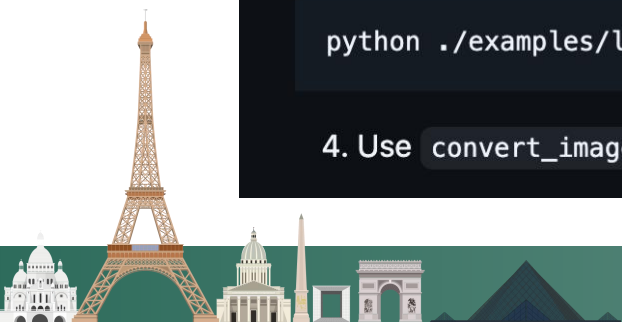
2. Install the required Python packages:

```
pip install -r examples/llava/requirements.txt
```

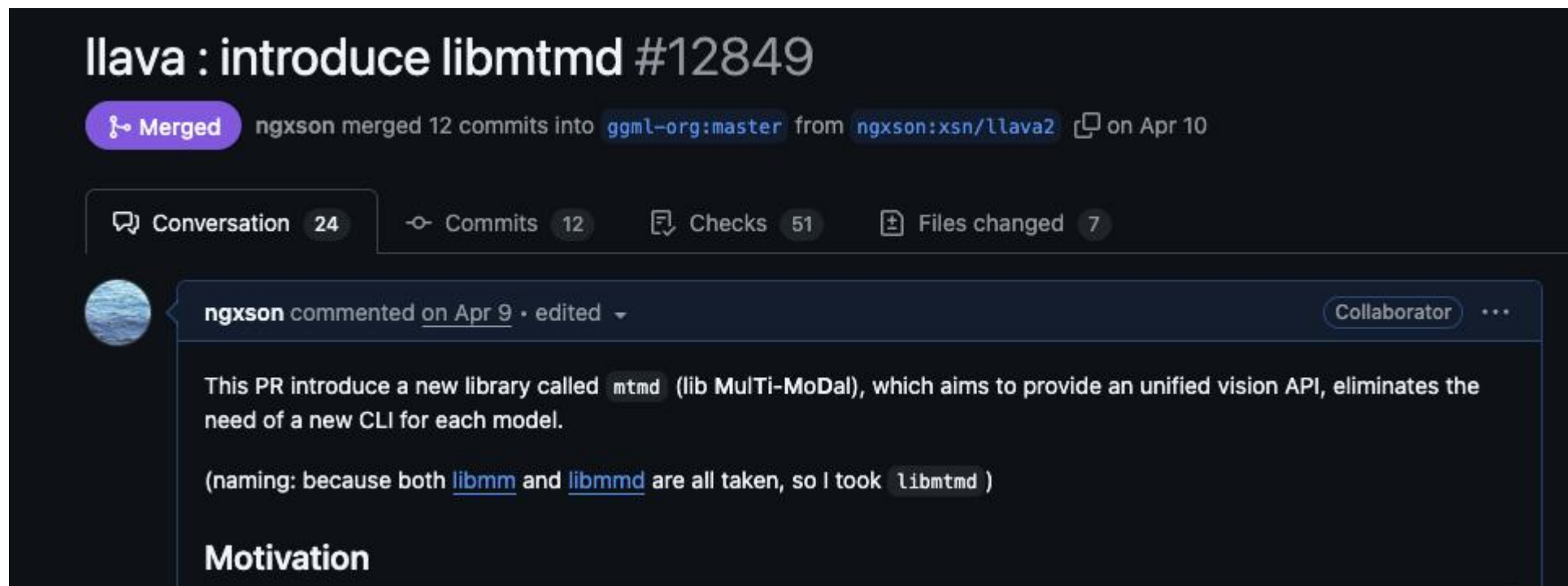
3. Use `llava_surgery.py` to split the LLaVA model to LLaMA and multimodal projector constituents:

```
python ./examples/llava/llava_surgery.py -m ../llava-v1.5-7b
```

4. Use `convert_image_encoder_to_gguf.py` to convert the LLaVA image encoder to GGUF:



→ Mid 2025, I introduced a sub-module inside llama.cpp called mtmd (multimodal)



The screenshot shows a GitHub pull request interface. At the top, the title is "llava : introduce libmtmd #12849". Below the title, a status bar indicates "Merged" in a purple pill, followed by the text "ngxson merged 12 commits into `ggml-org:master` from `ngxson:xsn/llava2` on Apr 10".

Below the status bar, there are tabs for "Conversation" (24), "Commits" (12), "Checks" (51), and "Files changed" (7). The "Conversation" tab is selected.

A comment by user "ngxson" is visible, dated "on Apr 9" and marked as "edited". The comment text reads: "This PR introduce a new library called `mtmd` (lib Multi-MoDal), which aims to provide an unified vision API, eliminates the need of a new CLI for each model. (naming: because both `libmm` and `libmmd` are all taken, so I took `libmtmd`)". The word "Collaborator" is visible next to the comment.

Below the comment, the word "Motivation" is visible, indicating the start of the motivation section.



Mtmd aims for:

- Stable API for multimodal processing
- Support multiple modalities
- Easy for developers and for users

More technical details?

Watch my talk at GOSIM 2025:

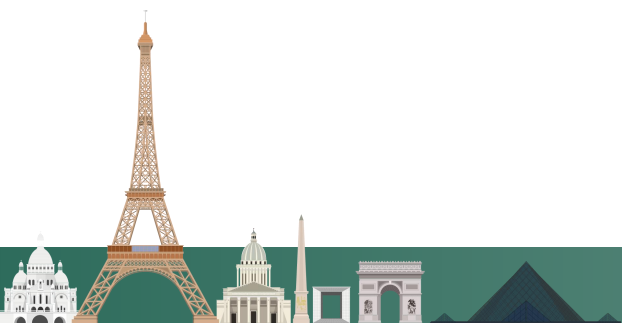
<https://www.youtube.com/watch?v=D3qISFta4Hc>



(Demo in next section)



Youtube video



Support modalities: audio input, vision input
Work-in-progress: audio output, video input
OCR / ASR capabilities

Ref OCR blog post: <https://huggingface.co/blog/ggml-org/using-ocr-models-with-llama-cpp>



A screenshot of the Hugging Face website showing a blog post. The header includes the Hugging Face logo, a search bar, and navigation links for Models, Datasets, Spaces, Buckets, Docs, and Pricing. Below the header is a login prompt. The main content area shows the article title 'Using OCR models with llama.cpp', a 'Community Article' badge, the publication date 'Published April 10, 2026', and an 'Edit article' button. The author's profile 'Xuan-Son Nguyen' is shown with their GitHub handle 'ngxson' and website 'ggml-org'. On the right, there is a sidebar with 'Upvoted 27' (with 15 more icons), 'Collections mentioned in this article 1', and a collection titled 'OCR models' with 10 items, updated 17 days ago.



02

Server, Web UI and CLI



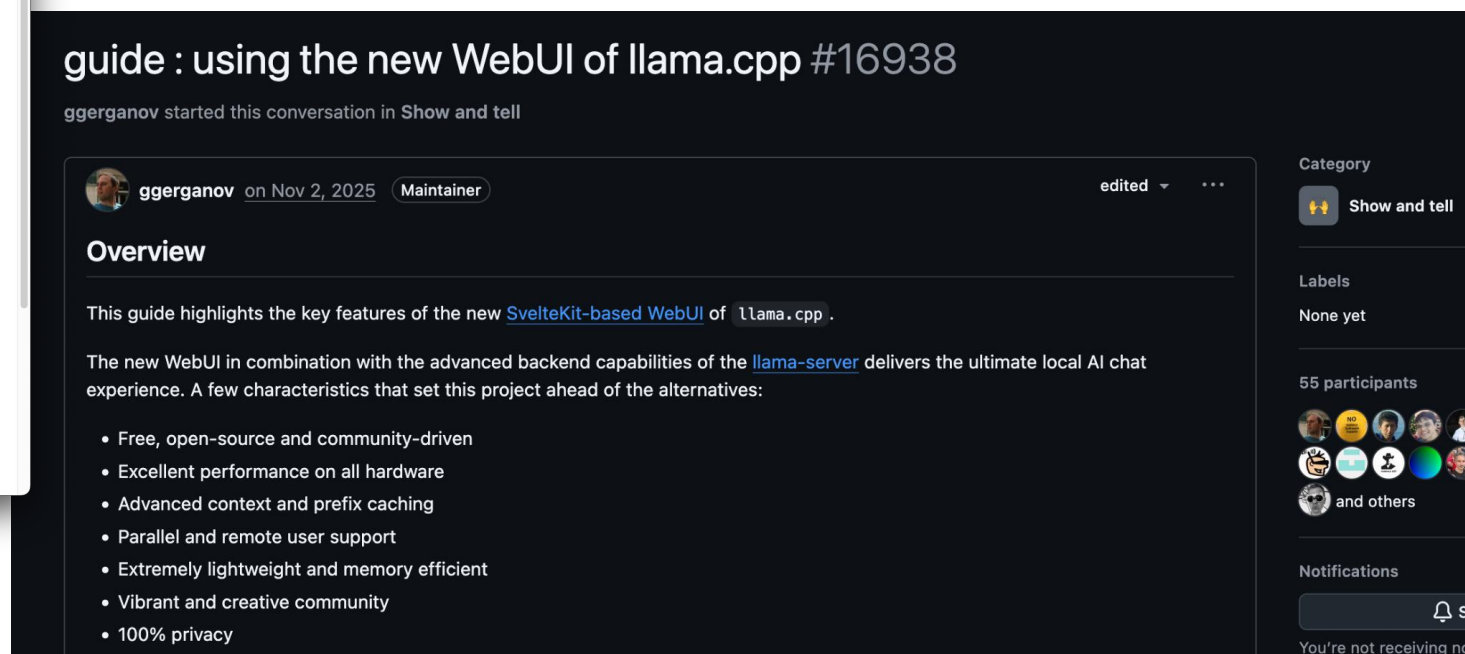
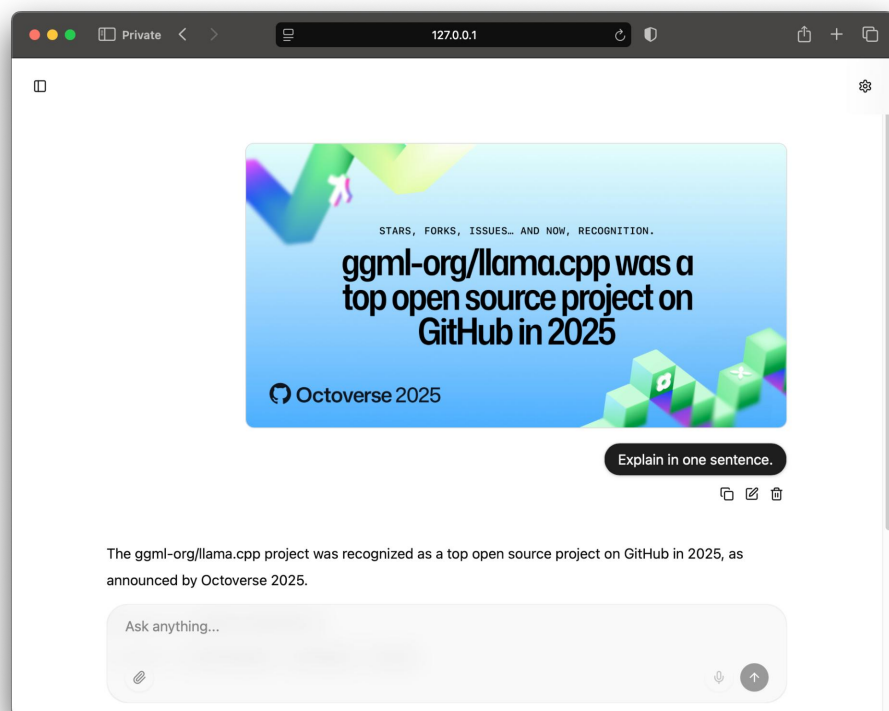
SERVER

- Multimodal support via openai-compatible api
- Streamed reasoning and tool calling (+ parallel tool calls) possible (thanks to @pwilkin and @aldehir for chat parsing infrastructure)
- Anthropic-compatible api: <https://huggingface.co/blog/ggml-org/anthropic-messages-api-in-llamacpp>
- Server “router” mode allow using multiple models via the same llama-server: <https://huggingface.co/blog/ggml-org/model-management-in-llamacpp>



New Web UI overhaul thanks to Alek (@allozaur)

Link to guide: <https://github.com/ggml-org/llama.cpp/discussions/16938>



New CLI, simple to use; Example command:

```
llama-cli -hf ggml-org/gemma-4-26B-A4B-it-GGUF
```

```
llama.cpp

build      : b7319-fa95df053
model      : Qwen3-8B-Q4_K_M.gguf
modalities : text

available commands:
  /exit or Ctrl+C  stop or exit
  /regen           regenerate the last response
  /clear           clear the chat history
  /read            add a text file

> hi, how are you

[Start thinking]
Okay, the user greeted me with "hi, how are you." I need to respond appropriately. First, I should acknowledge their greeting and express that I'm here to help. Since I'm an AI, I don't have feelings, but I can mention that I'm functioning well. I should keep the tone friendly and open-ended to encourage them to ask questions. Let me make sure the response is concise and welcoming. Maybe add an emoji to keep it friendly. Let me check for any typos or errors. Yeah, that should work.
[End thinking]

Hello! I'm just a virtual assistant, so I don't have feelings, but I'm here and functioning well! How can I assist you today? 😊

[ Prompt: 118.2 t/s | Generation: 47.3 t/s ]

>
```

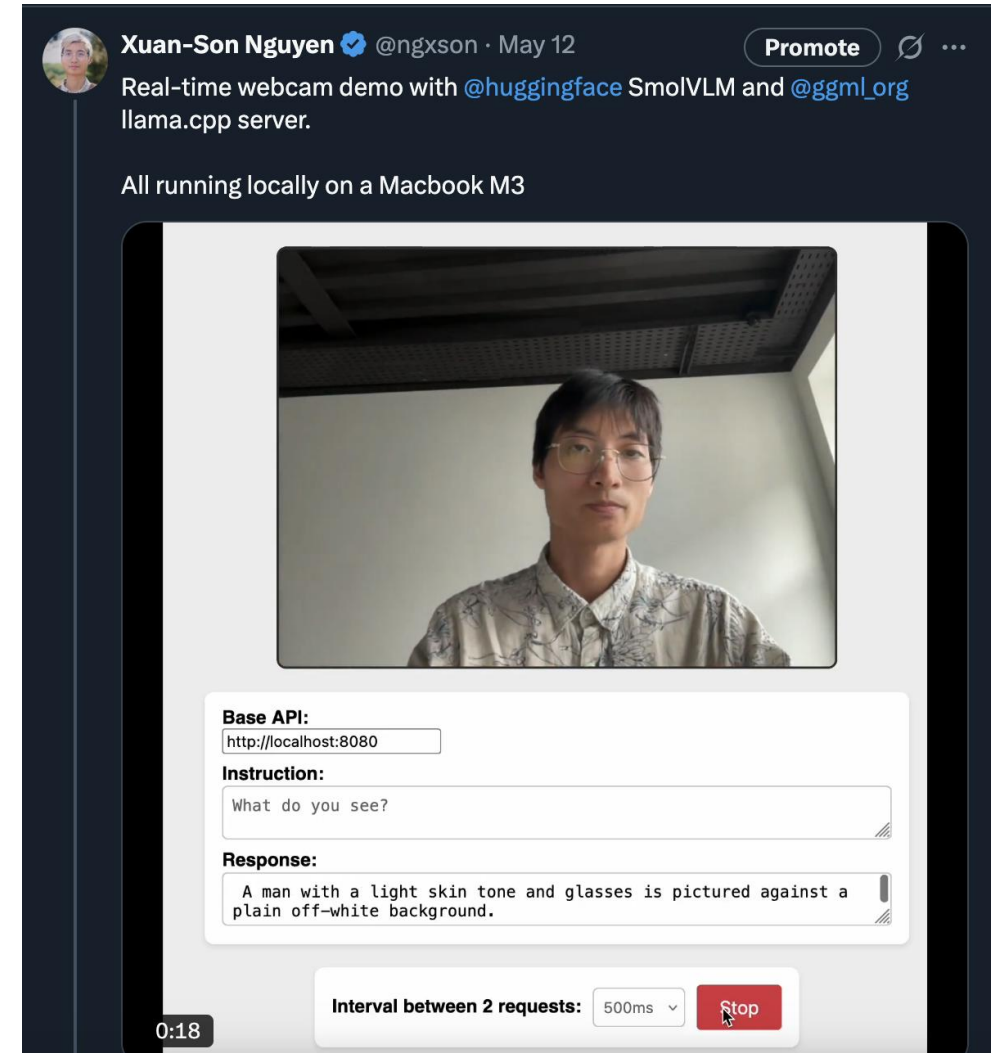


Demo Web UI / server:

- Start a model
- Upload image
- Watch the API calls
- MCP servers



See more: Real-time webcam recognition
<https://x.com/ngxson/status/1921980096421806127>



03

Performance improvements

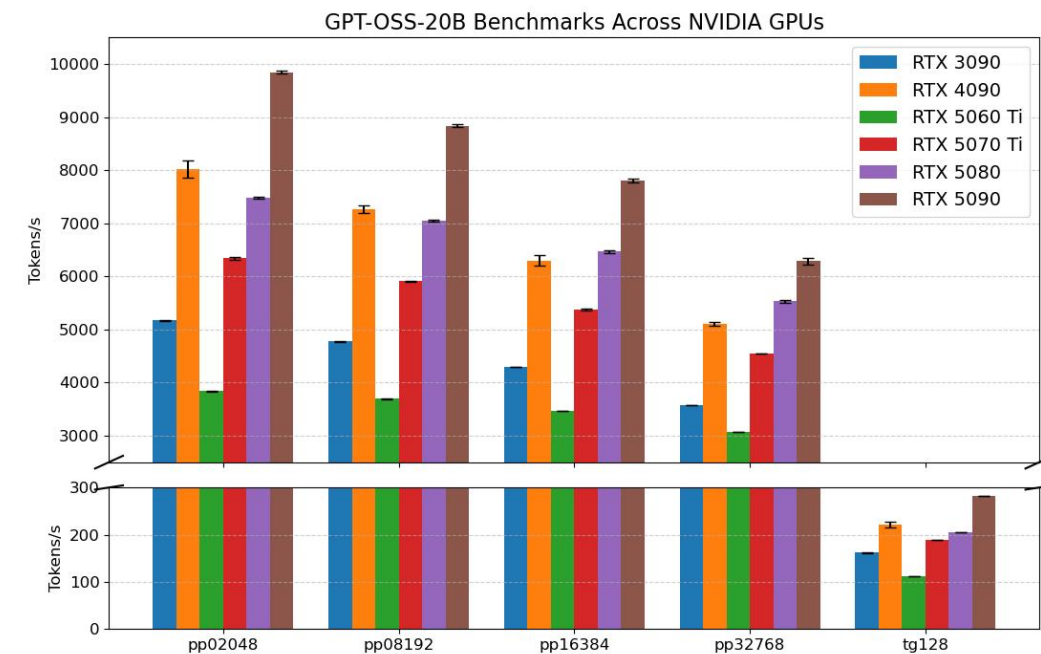


Many backend improvements, I cannot list all - most recently:

- Various improvements in Flash Attention kernel across all backends
- Cross-backend tensor parallelism (thanks to @JohannesGaessler)
- Improved support for multi-GPU setup
- New backends: WebGPU, VirtGPU, ZenDNN
- MXFP4 and NVFP4 support

Example: GPT-OSS benchmarks:

<https://github.com/ggml-org/llama.cpp/discussions/15396>



Inference improvements:

- Self-speculative decoding (via n-gram)
<https://github.com/ggml-org/LlamaBarn/issues/83>
- Better context re-using for recurrence models via context checkpoints (Qwen 3.5 / Qwen 3.6)



 ggernanov opened 5 days ago Member ...

With recent advances in prompt-based speculative decoding in `llama.cpp`, there is a new option that enables a "default" setup for speculative decoding. The setup is compatible with all models and does not require extra resources. In certain use cases, it leads to significant performance improvements:

- Agentic workflows
- Editing files
- Summarization

Recommend to add the new `--spec-default` flag to the `llama-server` startup command. This flag was introduced in [ggml-org/llama.cpp#22223](https://github.com/ggml-org/llama.cpp#22223), so it requires the latest version of `llama.cpp`.

More info:

- [arg](#) : add `--spec-default llama.cpp#22223`
- [server](#) : speculative checkpointing [llama.cpp#19493](#)
- [spec](#) : add `ngram-mod llama.cpp#19164`



04

Models and quantizations



- Model SOTA models are supported, including: Qwen 3.x, Gemma 4, Kimi-K2.x, Mistral Minstral / Mistral Small, GPT-OSS, GLM family
- To search for compatible, go to Hugging Face and search for “(model name) GGUF”

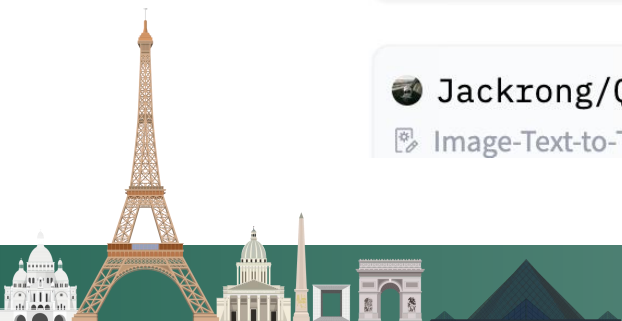
Models 21,311 Full-text search  Inference Available  Sort: Trending

 unsloth/Qwen3.6-27B-GGUF
 Image-Text-to-Text • 27B • Updated 5 days ago • 636k • 433

 unsloth/Qwen3.6-35B-A3B-GGUF
 Image-Text-to-Text • 35B • Updated 7 days ago • 1.65M • 806

 hesamation/Qwen3.6-35B-A3B-Claude-4.6-Opus-Reasoning-Distilled-GGUF
 Text Generation • 35B • Updated 8 days ago • 129k • 194

 Jackrong/Qwen3.6-27B-GGUF
 Image-Text-to-Text • 27B • Updated 5 days ago • 23.1k • 49



- New quantization: MXFP4 and NVFP4
- KV cache quantizations (Q4, Q8) with Hadamard transformation

llama : rotate activations for better quantization #21038

 Merged [ggerganov](#) merged 12 commits into `master` from `gg/attn-rot`  last month

 Conversation 51  Commits 12  Checks 47  Files changed 4

 **ggerganov** commented on Mar 26 · edited  Member 

Overview

In anticipation of the incoming flood of vibe generated PRs implementing [TurboQuant](#), I'm raising the baseline a bit using a very simple interpretation of the idea of using Hadamard transform to reduce outliers in the attention and improve the quantization quality:

- Rotate input Q, K, V and store in cache
- Perform attention with the rotated Q, K, V
- Rotate back the output vector of the attention

This works because:

- Rotation does not affect the dot product of 2 vectors
- The output vector is a linear combination of rotated vectors

Reviewers

-  CISC
-  pwilkin

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone



05

Future plans



- Add Agent mode to CLI and Web UI
- Multimodal: video input, audio output
(Longer in the future, maybe add image output)
- Multi-token prediction support
- Backend and speed optimizations; quantization improvements...



Thanks!



My website

